

VIETNAMESE-GERMAN UNIVERSITY  
Computer Science and Engineering

## COURSE PROJECT REPORT

**Course Title:** Real-Time Systems

*Smart Controller System using RTOS*

### Group Members

| No. | Student ID | Full Name            | Role / Tasks                               |
|-----|------------|----------------------|--------------------------------------------|
| 1   | 10423100   | Nguyễn Ngọc Bình Sơn | Code Writer (Semaphore), Report Writer     |
| 2   | 10423116   | Nguyễn Đức Trung     | Code Writer (Blinking task), Report Writer |

Ho Chi Minh city, 23<sup>rd</sup> July 2026

## Project Requirements

---

**Objective:** Develop an RTOS-based Smart Climate Control System that monitors temperature and humidity, and controls a heater, cooler, and humidifier (represented by LEDs) to maintain target conditions. The project is a course demo and need not reflect real-world physical operation.

**Hardware:** YoluUNO (ESP32-S3) microcontroller; DHT20 temperature/humidity sensor on I2C; three two-color LED actuators representing Heater (D3/D4), Cooler (D5/D6), and Humidifier (D7/D8); onboard status LED on GPIO48.

**RTOS:** FreeRTOS (or any RTOS suitable for the microcontroller), using the task scheduler.

**Required behaviors:** Serial print of temperature/humidity every 5 seconds; onboard LED blinks every 1 second; Heater LED shows GREEN/ORANGE/RED according to 3 defined temperature thresholds; Cooler activates for a fixed duration (e.g., 5s) when temperature exceeds a threshold, then re-checks; Humidifier runs a GREEN (5s) → YELLOW (3 s) → RED (2s) sequence when humidity falls below a threshold, then re-checks humidity to decide whether to repeat.

**Required tasks:** Blinky Task, Read Temperature Task, Cooler Task, Heater Task, Humidifier Task.

**Deliverable:** GitHub repository link and result snapshots.

# 1. Introduction

---

## 1.1 Problem Description

This project implements a small real-time climate controller on the OhStemApp Yolo UNO simulator. The DHT20 sensor provides the temperatures and relative humidity. The LCD and serial monitor show the current sample, while three RGB LED modules represent the heater, cooler, and humidifier outputs.

The system is intentionally written as an RTOS-style concurrent program instead of one long sequential loop. Each hardware behavior has its own task, and the sensor task distributes a ClimatePacket object through queues. This keeps timing independent: the heartbeat LED can blink every second while the humidifier sequence is still running and while future sensor samples are being produced.

## 1.2 Required Features

- Read temperature and humidity from DHT20 every 5 seconds.
- Print the measured values and controller decisions to the serial monitor.
- Show temperature, humidity, heater state, cooler state, and humidifier state on LCD1602.
- Blink the onboard D13 LED every 1 second as a system-alive indicator.
- Control heater LED states by thresholds: green SAFE, orange WARNING, red CRITICAL.
- Activate cooler LED for a fixed 5 second duration when temperature is above 30°C.
- Run humidifier LED sequence green 5s, yellow 3s, red 2s when humidity is below 40%.

## 1.3 Additional Proposed Features (Optional)

- Queue-only sensor sharing: no global temperature or humidity variables are used.
- A ClimatePacket object carries multiple fields in one queue message, matching the lecturer's advice to use a structure/object with several data fields.
- The RTOSQueue class uses a counting semaphore for availability and a binary semaphore as a mutex for queue integrity.
- Separate io\_mutex and actuator\_mutex make critical sections explicit for serial/LCD output and RGB LED driver access.

## 2. System Design using State Machine

### 2.1 Overall System State Machine



The flow starts at the initial node and enters **Initialize System**, which sets up config, pins, sensors (DHT20), LEDs, LCD, mutexes, and queues. Once setup completes (setUpComplete), control moves to **Scheduler Running**, which manages coroutine tasks and blinks a status LED. When a sample is due (sampleReady), it transitions to **Read Climate**, where the DHT20 sensor reads temperature and humidity. Once data is ready (dataReady), it flows into **Build And Dispatch Packet**, which packages the readings into a ClimatePacket, determines heater state, and pushes it onto all output queues.

From there (queuesReady), the **Display Task** state picks up the packet and safely renders it to the LCD. After displaying (displayDone), the flow hits a fork into three parallel branches: **Cooler Control**, **Heater Indication**, and **Humidifier Sequence** — each pulling from its own queue and driving its respective RGB LED/actuator based on the packet's data. These three branches then join at a synchronization bar (cycleComplete), which loops back to **Initialize system**, restarting the cycle.

Separately, from **Scheduler Running**, there's a **Shut down** transition leading directly to the final (terminal) state, ending the state machine.

## 2.2 Heater Control State Machine

| State    | Condition                                                                                            | Action                                  |
|----------|------------------------------------------------------------------------------------------------------|-----------------------------------------|
| SAFE     | $22^{\circ}\text{C} \leq T \leq 28^{\circ}\text{C}$                                                  | D3.D4 RGB = green                       |
| WARNING  | $18^{\circ}\text{C} \leq T < 22^{\circ}\text{C}$ or $28^{\circ}\text{C} < T \leq 32^{\circ}\text{C}$ | D3.D4 RGB = orange/yellow warning color |
| CRITICAL | $T < 18^{\circ}\text{C}$ or $T > 32^{\circ}\text{C}$                                                 | D3.D4 RGB = red                         |

## 2.3 Cooler Control State Machine

| State   | Transition                                                    | Action                                          |
|---------|---------------------------------------------------------------|-------------------------------------------------|
| IDLE    | packet.cooler_on is False because $T \leq 30^{\circ}\text{C}$ | D5.D6 RGB is off                                |
| COOLING | packet.cooler_on is True because $T > 30^{\circ}\text{C}$     | D5.D6 RGB turns green for 5s                    |
| RECHECK | After 5s delay finishes                                       | LED turns off; task waits for next queue packet |

## 2.4 Humidifier Control State Machine

| State   | Duration / condition                                        | Action                              |
|---------|-------------------------------------------------------------|-------------------------------------|
| IDLE    | packet.humidifier_on is False because $\text{RH} \geq 40\%$ | D7.D8 RGB is off                    |
| GREEN   | $\text{RH} < 40\%$ , first 5s                               | Humidifier starts                   |
| YELLOW  | Next 3s                                                     | Intermediate warning step           |
| RED     | Final 2s                                                    | End of cycle                        |
| RECHECK | After 10s total sequence                                    | LED off; wait for next queue packet |

### 3. System Implementation

#### 3.1 Semaphore and Synchronization Strategy

The lecturer required queues instead of global variables. The design therefore uses ClimatePacket objects and queue message passing. A packet contains sequence, temperature, humidity, heater\_state, cooler\_on, and humidifier\_on. Each consumer task receives the same packet but only processes the fields relevant to its job.

| RTOS object        | Code object                                                              | Purpose                                                                                          |
|--------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Counting semaphore | <code>RTOSQueue.item_ready = Semaphore(0)</code>                         | Blocks <code>get()</code> until the producer has put a packet into the queue.                    |
| Mutex              | <code>RTOSQueue.mutex = Semaphore(1)</code>                              | Protects list append/pop so queue operations are atomic.                                         |
| Mutex              | <code>io_mutex = Semaphore(1)</code>                                     | Prevents serial print and LCD writes from overlapping between tasks.                             |
| Mutex              | <code>actuator_mutex = Semaphore(1)</code>                               | Serializes RGB LED driver writes.                                                                |
| Queue              | <code>display_queue, heater_queue, cooler_queue, humidifier_queue</code> | Sends ClimatePacket messages from sensor task to consumer tasks without global sensor variables. |

```
class Semaphore:
    def __init__(self, value=1):
        if value < 0:
            raise ValueError('ValueError')
        self.value = value
        self.waiting = []

    async def acquire(self):
        token = object()
        self.waiting.append(token)

        while self.value <= 0 or self.waiting[0] is not token:
            await asleep_ms(10)

        self.waiting.pop(0)
        self.value -= 1
        return True

    def release(self):
        self.value += 1

class RTOSQueue:
    def __init__(self, max_items=5):
        self.queue = []
```

```
self.max_items = max_items
self.item_ready = Semaphore(0)
self.mutex = Semaphore(1)

async def put(self, item):
    await self.mutex.acquire()
    try:
        if len(self.queue) >= self.max_items:
            self.queue.pop(0)
        else:
            self.item_ready.release()
            self.queue.append(item)
    finally:
        self.mutex.release()

async def get(self):
    await self.item_ready.acquire()
    await self.mutex.acquire()
    try:
        return self.queue.pop(0)
    finally:
        self.mutex.release()
```

### 3.2 Blinky Task

Priority/period: low priority periodic heartbeat. It toggles LED D13 every 1000ms and does not depend on sensor data.

```
async def task_LED_Blinky(led_D13):
    while True:
        led_D13.toggle()
        await asleep_ms(1000)
```

### 3.3 Read Temperature Task

Priority/period: periodic producer task every 5000ms. It reads DHT20, computes all control decisions once, creates a ClimatePacket, and puts that object into every task queue.

```
async def task_Read_Temperature(dht20, queue_list, config):
    sequence = 0

    while True:
        sequence += 1
        temperature = await dht20.atemperature()
        humidity = await dht20.ahumidity()

        packet = ClimatePacket(
            sequence,
            temperature,
            humidity,
            get_heater_state(temperature, config),
            temperature > config.cooler_temp_on,
```

```
        humidity < config.humidity_min
    )

    await add_packet_to_all_queues(packet, queue_list)
    await asleep_ms(5000)
```

### 3.4 Cooler Task

Trigger: cooler\_queue.item\_ready semaphore. It only uses packet.temperature and packet.cooler\_on. If cooler\_on is true, the cooler LED stays green for 5s, then turns off and waits for the next queued packet.

```
async def task_Cooler(cooler_queue, rgb_led_D5, actuator_mutex,
io_mutex, config):
    while True:
        packet = await cooler_queue.get()

        if packet.cooler_on:
            await safe_print(io_mutex, 'Cooler Task: ON for 5s at
{:.1f}C'.format(packet.temperature))
            await safe_rgb_show(actuator_mutex, rgb_led_D5,
config.color_green)
            await asleep_ms(5000)
            await safe_rgb_show(actuator_mutex, rgb_led_D5,
config.color_off)
            await safe_print(io_mutex, 'Cooler Task: recheck after fixed
duration')
        else:
            await safe_rgb_show(actuator_mutex, rgb_led_D5,
config.color_off)
            await safe_print(io_mutex, 'Cooler Task: OFF at
{:.1f}C'.format(packet.temperature))
```

### 3.5 Heater Task

Trigger: heater\_queue.item\_ready semaphore. It only uses packet.heater\_state and packet.temperature. SAFE maps to green, WARNING maps to orange/yellow, and CRITICAL maps to red.

```
async def task_Heater(heater_queue, rgb_led_D3, actuator_mutex,
io_mutex, config):
    while True:
        packet = await heater_queue.get()

        if packet.heater_state == 'SAFE':
            await safe_rgb_show(actuator_mutex, rgb_led_D3,
config.color_green)
        elif packet.heater_state == 'WARNING':
            await safe_rgb_show(actuator_mutex, rgb_led_D3,
config.color_orange)
        else:
```



```
        await safe_rgb_show(actuator_mutex, rgb_led_D3,
config.color_red)

        await safe_print(io_mutex, 'Heater Task: {} at
{:.1f}C'.format(packet.heater_state, packet.temperature))
```

### 3.6 Humidifier Task

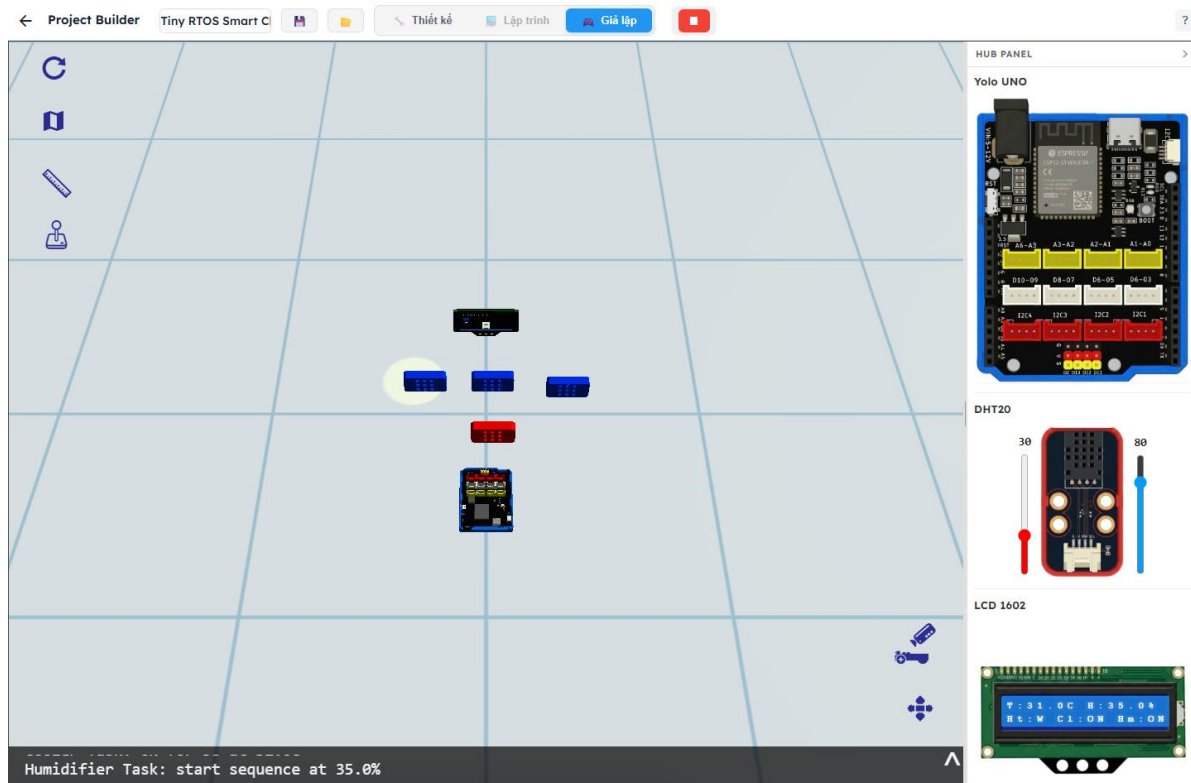
Trigger: humidifier\_queue.item\_ready semaphore. It only uses packet.humidity and packet.humidifier\_on. When humidity is below 40%, it executes the required green-yellow-red timed sequence and then waits for the next packet.

```
async def task_Humidifier(humidifier_queue, rgb_led_D7,
actuator_mutex, io_mutex, config):
    while True:
        packet = await humidifier_queue.get()

        if packet.humidifier_on:
            await safe_print(io_mutex, 'Humidifier Task: start sequence at
{:.1f}%'.format(packet.humidity))
            await safe_rgb_show(actuator_mutex, rgb_led_D7,
config.color_green)
            await asleep_ms(5000)
            await safe_rgb_show(actuator_mutex, rgb_led_D7,
config.color_yellow)
            await asleep_ms(3000)
            await safe_rgb_show(actuator_mutex, rgb_led_D7,
config.color_red)
            await asleep_ms(2000)
            await safe_rgb_show(actuator_mutex, rgb_led_D7,
config.color_off)
            await safe_print(io_mutex, 'Humidifier Task: sequence done, wait
next queue packet')
        else:
            await safe_rgb_show(actuator_mutex, rgb_led_D7,
config.color_off)
            await safe_print(io_mutex, 'Humidifier Task: OFF at
{:.1f}%'.format(packet.humidity))
```

## 4. Validation Results in Simulation (or Hardware)

### 4.1 Monitoring Output



Simulation input was seeded to temperature = 31°C and humidity = 35%. The LCD output read T:31.0°C H:35.0% and Ht:W Cl:ON Hm:ON, proving the display task received a queued packet and used it correctly.

| Verification item               | Result |
|---------------------------------|--------|
| Yolo UNO present                | Pass   |
| DHT20 on I2C1                   | Pass   |
| LCD1602 on I2C2                 | Pass   |
| RGB LEDs on D3.D4, D5.D6, D7.D8 | Pass   |
| No fatal runtime logs           | Pass   |
| LCD shows sensor values         | Pass   |

Observed serial output: Tiny RTOS Smart Climate Controller started | Queue design: ClimatePacket object is copied into task queues | Semaphore design: item\_ready counts queue

data, Semaphore(1) works as mutex | Sample 1: temp=31.0°C humidity=35.0% heater=WARNING cooler=ON humidifier=ON | Heater Task: WARNING at 31.0°C | Cooler Task: ON for 5s at 31.0°C | Humidifier Task: start sequence at 35.0%

#### **4.2 Heater Behavior**

At  $T = 31^{\circ}\text{C}$ , the rule  $28^{\circ}\text{C} < T \leq 32^{\circ}\text{C}$  places the heater display in WARNING. The simulator showed the heater output active in the warning color and the serial monitor printed: Heater Task: WARNING at 31.0°C.

#### **4.3 Cooler Behavior**

At  $T = 31^{\circ}\text{C}$ , packet.cooler\_on is True because  $31^{\circ}\text{C} > 30^{\circ}\text{C}$ . The cooler task printed 'Cooler Task: ON for 5s at 31.0°C', set the D5.D6 RGB LED to green, delayed for 5000 ms, and then turned the LED off before waiting for another queued packet.

#### **4.4 Humidifier Behavior**

At  $\text{RH} = 35\%$ , packet.humidifier\_on is True because  $35\% < 40\%$ . The humidifier task printed the start message and executed green for 5s, yellow for 3s, red for 2s, then off. This is independent from the sensor task because it runs in a separate task and receives its own queued packet.

#### **4.5 Source Code Repository**

GitHub link: <https://github.com/NguyenDucTrung21/Smart-Controller-System-using-RTOS.git>

OhStemApp link: <https://app.ohstem.vn/#!/share/vrb-project/a903bf0c-3d3c-443d-a328-82c918c270c8>

## 5. Conclusion and Perspective

---

### 5.1 Summary of Achievements

- The completed OhStem project uses the exact required hardware: Yolo UNO, DHT20, LCD1602, and three RGB4LED actuator modules.
- The code runs as independent asynchronous RTOS-style tasks: LED blinky, sensor producer, display, heater, cooler, and humidifier.
- Sensor information is transferred through queues as ClimatePacket objects instead of global variables.
- Semaphores and mutexes are explicitly implemented and used for queue blocking, queue protection, shared I/O, and RGB actuator access.
- The final share link was verified in the OhStem simulation tab: <https://app.ohstem.vn/#!/share/vrb-project/8cff11d5-6fa7-4801-bce6-fd1290f59c64>

### 5.2 Limitations

- The simulator represents actuators using LEDs, not real heater, cooler, or humidifier hardware.
- The controller uses threshold logic rather than PID or hysteresis; this is suitable for the assignment but could switch frequently near thresholds in real hardware.
- The custom Semaphore and RTOSQueue are written in MicroPython/asyncio style for OhStemApp simulation rather than using direct FreeRTOS C APIs.

### 5.3 Future Work

1. Add hysteresis bands, for example cooler ON above 30°C and OFF below 28°C, to avoid rapid toggling.
2. Add real actuator drivers and safety interlocks before applying the design to physical heating/cooling equipment.
3. Store timestamped ClimatePacket samples for trend display, alarms, or a web dashboard.